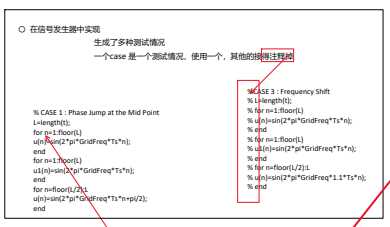
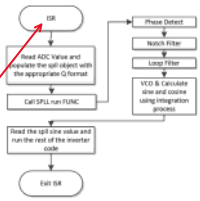
锁相环模拟测试

2020年4月17日 10:53

matlab代码解析

2020年4月17日 10:54





整个过程重复执行框图中的操作。在AD采集完成的中断服务程序中。 仿真只重复有限次数Tfinal/Ts，且（如果数据够多）n从几开始都没关系

% simulate the PLL process

for n=2:Tfinal/Ts % No of iteration of the PLL process in the simulation time

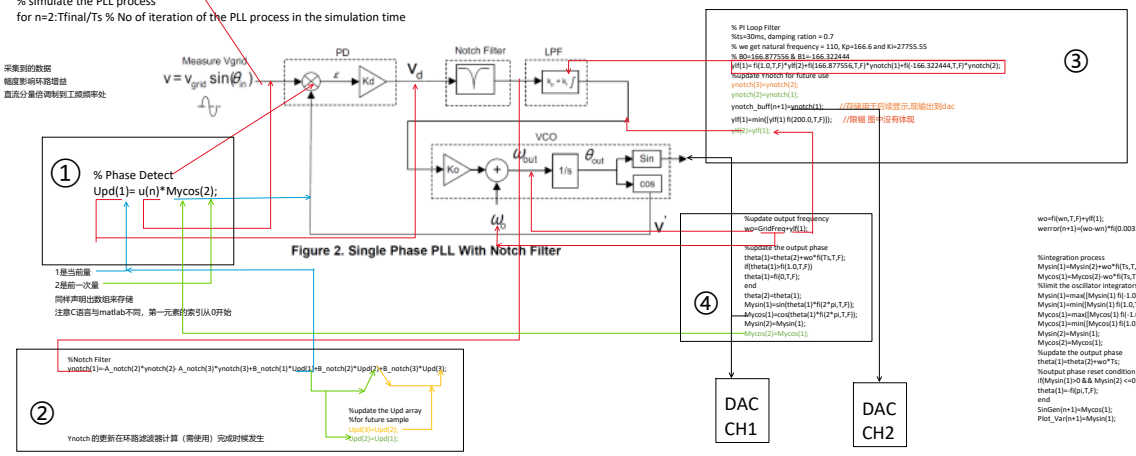


Figure 2. Single Phase PLL With Notch Filter

```
wo=fi(wn,T,F)+yff(1);
werror(n+1)=(wo-wn)*fi(0.00318309886,T,F);

%integration process
Mysin(1)=Mysin(2)+wo*fi(Ts,T,F)*(Mycos(2));
Mycos(1)=Mycos(2)-wo*fi(Ts,T,F)*(Mysin(2));
%limit the oscillator integrators
Mysin(1)=max([Mysin(1) fi(-1.0,T,F)]);
Mysin(1)=min([Mysin(1) fi(1.0,T,F)]);
Mycos(1)=max([Mycos(1) fi(-1.0,T,F)]);
Mycos(1)=min([Mycos(1) fi(1.0,T,F)]);
Mysin(2)=Mysin(1);
Mycos(2)=Mycos(1);
%update the output phase
theta(1)=theta(2)+wo*Ts;
%output phase reset condition
if(Mysin(1)>0 && Mysin(2) <=0)
theta(1)=-fi(pi,T,F);
end
SinGen(n+1)=Mycos(1);
Plot_Var(n+1)=Mysin(1);
```

Sine and Cosine Generation
The PLL uses sin and cos calculation, these calculations can consume large number of cycles in a typical microcontroller. To avoid this issue, the sine and cosine value is generated in this module by applying the principle of integration.

the coefficients for the notch filter can be adaptively changed as the grid frequency varies by calling a routine in the background that estimates the coefficients based on measure grid frequency.

```
%notch filter design
c1=0.1; c2=0.00001; X=2*c2*wn*2*Ts; Y=2*c1*wn*2*Ts; Z=wn*2*wn*2*Ts*Ts;
B_notch=[1 (X-2) (-X+Z+1)]; A_notch=[1 (Y-2) (-Y+Z+1)];
B_notch=fi(B_notch,T,F); A_notch=fi(A_notch,T,F);
```

Using Pre-Computation With the DF13 Controller

file:///E:/mcu/新建文件夹/C2000™%20Digital%20Controller%20Library%20users%20Guide.pdf

In control applications, the time between sampling the feedback and generating a corrective output is known as "sample-to-output delay". This delay imparts a phase lag into the loop that, in general, degrades performance. The computation time of the controller (and therefore the phase lag) can be reduced by precomputing those terms in the difference equation, which is already known when the new input sample arrives. The controller need then only compute one partial product, regardless of the order of the controller.

CMSIS-DSP library 浮点运算

2020年4月17日 17:17

http://www.keil.com/pack/doc/cmsis/DSP/html/group__sin.html#gae164899c4a3fc0e946dc5d55555fe541

arm_sin_cos_f32(float32_t theta, float32_t * pSinVal, float32_t * pCosVal);

Computes the trigonometric sine and cosine values using a combination of table lookup and linear interpolation.

float32_t arm_sin_f32 (float32_t x)

Parameters
[in] x input value in radians.

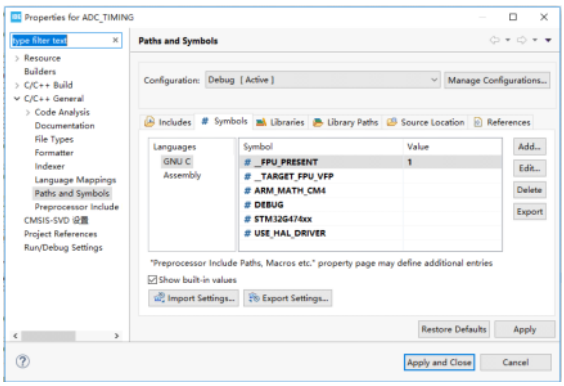
Returns
sin(x)

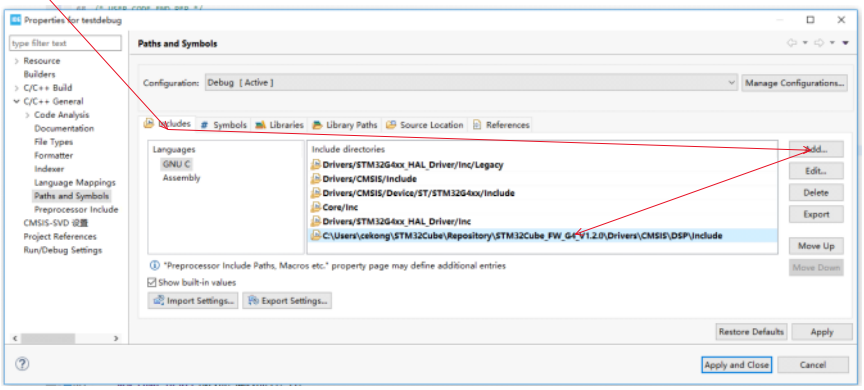
void arm_float_to_q15 (const float32_t *pSrc, q15_t *pDst, uint32_t blockSize)
Converts the elements of the floating-point vector to Q15 vector.

CMSIS DSP LIB ENABLE in STM32CUBEIDE

2020年4月17日 22:35

复制了三个头文件，源自
Or
添加相应的路径





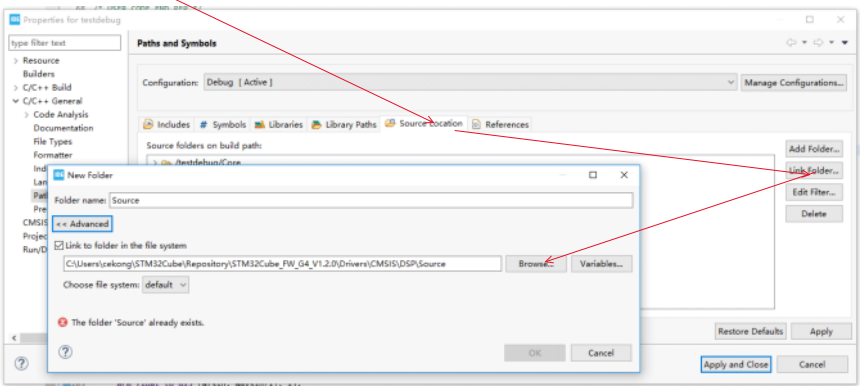
__FPU_PRESENT ，且数值1

__TARGET_FPU_VFP

ARM_MATH_CM4

编译成功

链接相应源文件



测试代码，生成了一个正弦表

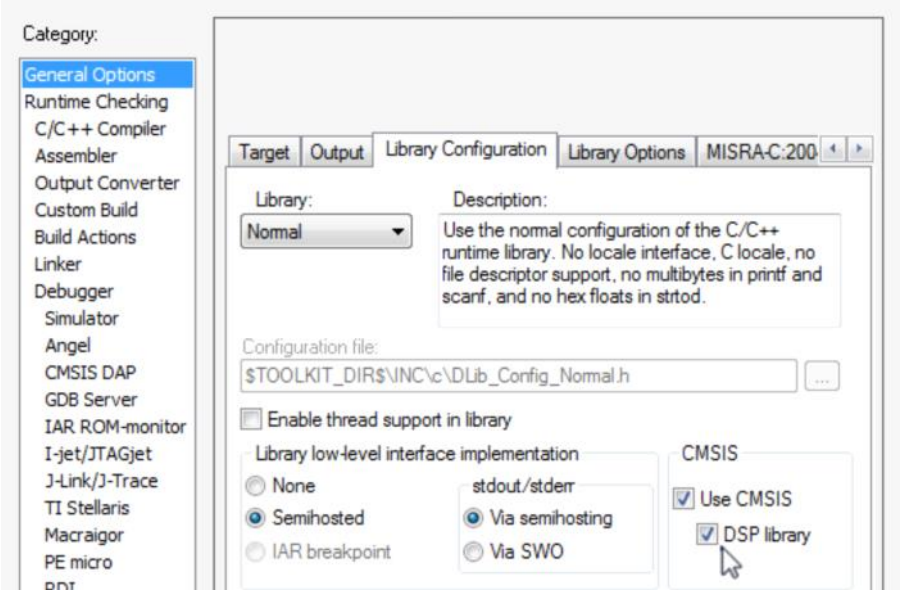
```
/* USER CODE BEGIN 0 */
#include <arm_math.h>
float32_t fsin,fcos;
float32_t fttheta;
float32_t famp =0.9;
#define pi 3.141592653
float32_t fthetstep = 2*pi*50/3000;
int16_t mysin[60];



/* USER CODE END 0 */

/**
  * @brief  The application entry point.
  * @retval int
  */
int main(void)
{
  /* USER CODE BEGIN 1 */

	for (int i = 0; i < 60; i++) {
		fttheta += fthetstep;

	if (fttheta > 2 * pi)
		fttheta = 0;
	arm_sqrt_f32(fttheta, &fsin);

	//arm_sin_cos_f32(  fttheta, & fsin,  &fcos);

	 fsin = arm_sin_f32( fttheta	);
	 arm_mult_f32 (&fsin, &famp, &fsin, 1);
	 arm_float_to_q15 (&fsin, &mysin[i], 1);
	}
```

CMSIS-DSP library with IAR Embedded Workbench

2022年4月3日 6:04

you enable the use of the CMSIS-DSP library by first choosing a Cortex-M device,

Second, set the CMSIS-DSP library option in the General Options>Library Configuration page. This will set the PATH for C preprocessor and import the pre-build CMSIS library.



These settings are all you need to be able to use CMSIS-DSP from IAR Embedded Workbench for ARM.

信号源

2020年4月18日 15:51

dac原理

2020年4月10日 16:33

```
#define DAC_CR_EN1_Pos          (0U)
#define DAC_CR_EN1_Msk          (0x1UL << DAC_CR_EN1_Pos)          /*!< 0x00000001 */
#define DAC_CR_EN1              DAC_CR_EN1_Msk                     /*!<DAC channel1 enable */

#define DAC_CHANNEL_1                    0x00000000U
#define DAC_CHANNEL_2                    0x00000010U    // 十六进制的0x10 十进制为 16

#define __HAL_DAC_ENABLE(__HANDLE__, __DAC_Channel__)\
  (((__HANDLE__)->Instance->CR) |= (DAC_CR_EN1 << (((__DAC_Channel__) & 0x10UL))))
```

如果是通道1 DAC_CR_EN1 <<0 不进行左移操作

如果是通道2 DAC_CR_EN1 <<16 左移16位（bit）

```
 /* Enable the Peripheral */
  __HAL_DAC_ENABLE(hdac, Channel);
```

Note:

The ENx bit enables the analog DAC channelx only. The DAC channelx digital interface is enabled even if the ENx bit is reset.

在有些mcu中，要方位相应外设的寄存器（ digital interface） 要先使能其时钟。

SET_BIT(hdac->Instance->SWTRIGR, DAC_SWTRIGR_SWTRIG1);

通过Instance 的decalaration 找到

```
typedef struct __DAC_HandleTypeDef
#else
typedef struct
#endif
{
  DAC_TypeDef                 *Instance;    /*!< Register base address             */
```

通过 DAC_TypeDef 的decalaration 找到

```
typedef struct
{
  __IO uint32_t CR;      /*!< DAC control register,                    Address offset: 0x00 */
  __IO uint32_t SWTRIGR;  /*!< DAC software trigger register,           Address offset: 0x04 */
-- --

} DAC_TypeDef;
```

定位 到数据手册中的寄存器说明

21.7.1 DAC control register (DAC_CR)

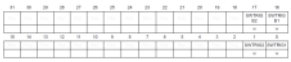
Address offset: 0x00

Reset value: 0x0000 0000

21.7.2 DAC software trigger register (DAC_SWTRGR)

Address offset: 0x04

Reset value: 0x0000 0000



This bit is cleared by hardware (one dac_hclk clock cycle later) once the DAC_DHR1 register value has been loaded into the DAC_DOR1 register.

```
#define DAC_CR_TSEL1_Pos        (2U)
#define DAC_CR_TSEL1_Msk        (0xFUL << DAC_CR_TSEL1_Pos)        /*!< 0x0000003C */  四个bit 左移2位
#define DAC_CR_TSEL1            DAC_CR_TSEL1_Msk                   /*!<TSEL1[3:0] (DAC channel1 Trigger selection ) */
```

```
#define DAC_CR_TEN1_Pos         (1U)
#define DAC_CR_TEN1_Msk         (0x1UL << DAC_CR_TEN1_Pos)         /*!< 0x00000002 */
#define DAC_CR_TEN1             DAC_CR_TEN1_Msk                    /*!<DAC channel1 Trigger enable */
```

The DHRx register is then loaded into the DORx register either automatically, by software trigger or by an external event trigger.

？ 这三种情况分别是如何设置，又应用在什么场合 21.4.5 DAC conversion

The DAC_DORx cannot be written directly and any data transfer to the DAC channelx must be performed by loading the DAC_DHRx register

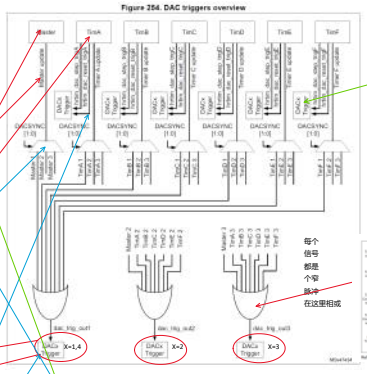
1、
if no hardware trigger is selected (TENx bit in DAC_CR register is reset).
DAC_DHRx register are automatically transferred to the DAC_DORx register after one dac_hclk clock cycle

2、
If the TENx control bit is set,
the conversion can then be triggered by an external event (timer counter, external interrupt line).
These events can be either the software trigger or hardware triggers， TSELx[3:0] control bits determine。
TSELx[3:0] bit cannot be changed when the ENx bit is set.

3、
If the software trigger is selected，（TSELx[3:0] = 0000: SWTRIG1）
the conversion starts（The DHRx register is then loaded into the DORx register） once the SWTRIG bit is set.

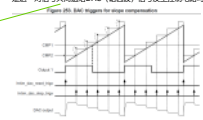
HRTIM 中 reset 与update 事件的区别与联系?

Figure 284. DAC triggers overview

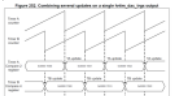


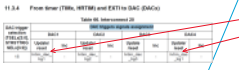
The update events from the master timer and the timer units can generate DAC update triggers on any of the 3 hrtim_dac_trgx outputs.

是这一对信号共同送给DAC（锯齿波）信号发生控制电路吗?



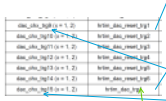
每个信号都是个窄脉冲在这里相或





An output pulse of 1 fHRTIM clock periods is generated on the hrtim_dac_trgx output.

为什么通常DAC都要精确（稳定）触发
对于快速变化的生成信号
触发不稳定，相当于在dac的输出中叠加了噪声



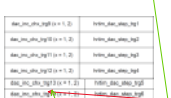
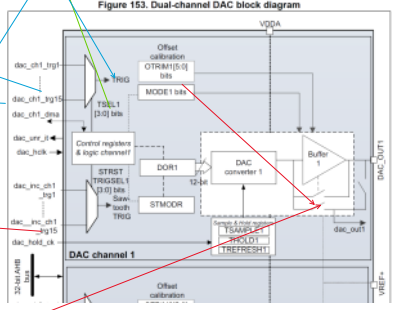


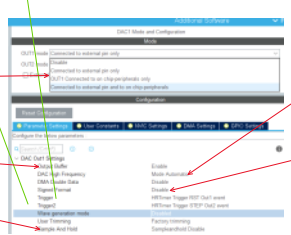
Figure 153. Dual-channel DAC block diagram



sConfig.DAC_HighFrequency = DAC_HIGH_FREQUENCY_INTERFACE_MODE_AUTOMATIC;

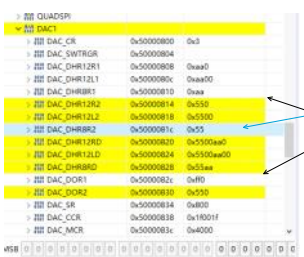
Table 177. HFSEL description

HFSEL[1:0]	AHB frequency	Function
00	≤ 80 MHz	DAC_DOR update rate up to 3 AHB clock cycles
01	>80 MHz[1]	DAC_DOR update rate up to 5 AHB clock cycles
10	>160 MHz	DAC_DOR update rate up to 7 AHB clock cycles
11	Reserved	-



Bits 17:16 MODE[2:0]: DAC channel2 mode
These bits can be written only when the DAC is disabled and not in the calibration mode (when bit EN2=0 and bit CEN2 =0 in the DAC_CR register). If EN2=1 or CEN2 =1 the write operation is ignored.
They can be set and cleared by software to select the DAC channel2 mode:
- DAC channel2 in Normal mode
000: DAC channel2 is connected to external pin with Buffer enabled
001: DAC channel2 is connected to external pin and to on chip peripherals with buffer enabled
010: DAC channel2 is connected to external pin with buffer disabled
011: DAC channel2 is connected to on chip peripherals with Buffer disabled
- DAC channel2 in Sample and hold mode:
100: DAC channel2 is connected to external pin with Buffer enabled
101: DAC channel2 is connected to external pin and to on chip peripherals with Buffer enabled
110: DAC channel2 is connected to external pin and to on chip peripherals with Buffer disabled
111: DAC channel2 is connected to on chip peripherals with Buffer disabled
Note: This register can be modified only when EN2=0

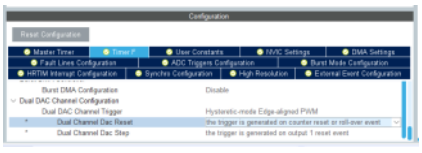
DAC mode control register (DAC_MCR)
Address offset: 0x3C
Bit 25 SINFORMAT2: Enable signed format for DAC channel2
This bit is set and cleared by software.
0: Input data is in unsigned format
1: Input data is in signed format (2's complement). The MSB bit represents the sign.

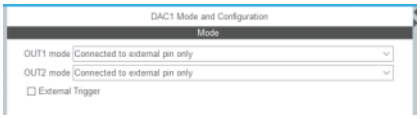


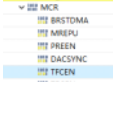
DAC内部DHR寄存器只有一个
为了方便编程，不同位数，对齐方式的多个可访问地址；
任意更新其中一个
相关联的寄存器
的值都发生了改变（debuger黄色提示）

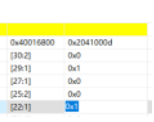
DOR2最之改变，是因为TEN位没有置位

Dac1 TEN 置位了，所以DOR1没变（没有触发信号到来）



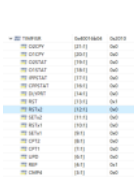




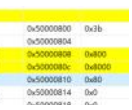


0x40016800	0x2041000d
[30:2]	0x0
[29:1]	0x1
[27:1]	0x0
[25:2]	0x0
[22:1]	0x1

HRTIM_TIMF		
TIMFCR	0x40016b00	0xd
TIMFISR	0x40016b04	0x2010
TIMFICR	0x40016b08	
TIMFDIER	0x40016b0c	0x0
CNTFR	0x40016b10	0x4a3

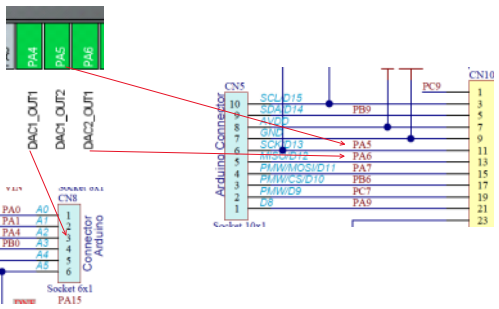


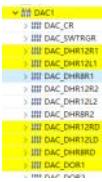
TIMFISR	0x40016b04	0x2010
CIDCPV	[31:1]	0x0
CIICPV	[30:1]	0x0
CIDCTAT	[19:1]	0x0
CIICTAT	[18:1]	0x0
IPRSTAT	[17:1]	0x0
CPPSTAT	[16:1]	0x0
SUPRT	[14:1]	0x0
RST	[13:1]	0x1
RSTx2	[12:1]	0x0
SETx2	[11:1]	0x0
RSTx1	[10:1]	0x0
SETx1	[9:1]	0x0
CPT2	[8:1]	0x0
CPT1	[7:1]	0x0
UPD	[6:1]	0x0
REP	[5:1]	0x1
CMP4	[4:1]	0x0

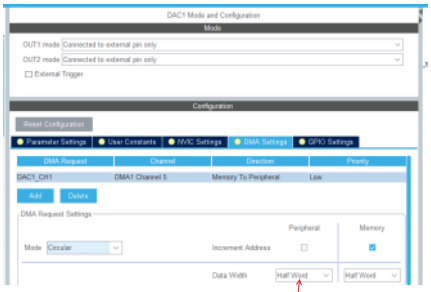


0x50000800	0x3b
0x50000804	
0x50000808	0x800
0x5000080c	0x8000
0x50000810	0x80
0x50000814	0x0

MCR	0x40016800	0x2001000d
BRSTDMA	[30:2]	0x0

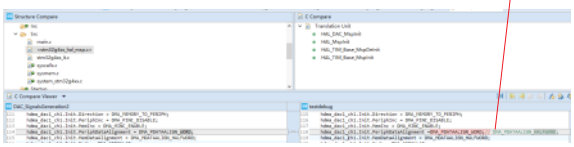






正常运行寄存器内容

DAC1			
DAC_CR	0x50000800	0x301f	0x301f
DAC_SWTRGR	0x50000804		0x3013
DAC_DHR12R1	0x50000808	0xff0	0xef4
DAC_DHR12L1	0x5000080c	0xff00	0xa5a0
DAC_DHR8R1	0x50000810	0xff	0xc7
DAC_DHR12R2	0x50000814	0x0	0x0
DAC_DHR12L2	0x50000818	0x0	0x0
DAC_DHR8R2	0x5000081c	0x0	0x0
DAC_DHR12RD	0x50000820	0xff0	0x994
DAC_DHR12LD	0x50000824	0xff00	0x7ff0
DAC_DHR8RD	0x50000828	0xff	0xe2
DAC_DOR1	0x5000082c	0xff0	0xd18
DAC_DOR2	0x50000830	0x0	0x0
DAC_SR	0x50000834	0x2800	0x800
DAC_CCR	0x50000838	0x1f001f	0x1f000e
DAC_MCR	0x5000083c	0x0	0x4000
DAC_SHSR1	0x50000840	0x0	0x0
DAC_SHSR2	0x50000844	0x0	0x0
DAC_SHHR	0x50000848	0x10001	0x10001
DAC_SHRR	0x5000084c	0x10001	0x10001
DAC_STR1	0x50000858	0x0	0x4441000
DAC_STR2	0x5000085c	0x0	0x0
DAC_STMODR	0x50000860	0x7	0x704



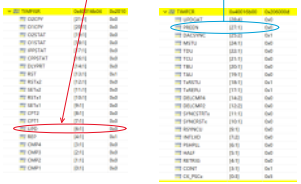
0x50000800	0x3b
0x50000804	
0x50000808	0x800
0x5000080c	0x8000
0x50000810	0x80
0x50000814	0x0
0x50000818	0x0
0x5000081c	0x0
0x50000820	0x800
0x50000824	0x8000
0x50000828	0x80
0x5000082c	0x800

MCR	0x40016800	0x2001000d
BRSTDMA	[30:2]	0x0
MREPU	[29:1]	0x1
PREEN	[27:1]	0x0
DACSYNC	[25:2]	0x0
TRGEN	[22:1]	0x0
DAC_SWTRGR	0x50000804	
DAC_DHR12R1	0x50000808	0x8f0
DAC_DHR12L1	0x5000080c	0x8f00
DAC_DHR8R1	0x50000810	0x8f
DAC_DHR12R2	0x50000814	0x0
DAC_DHR12L2	0x50000818	0x0
DAC_DHR8R2	0x5000081c	0x0
DAC_DHR12RD	0x50000820	0x8f0
DAC_DHR12LD	0x50000824	0x8f00
DAC_DHR8RD	0x50000828	0x8f
DAC_DOR1	0x5000082c	0x800
DAC_DOR2	0x50000830	0x0

PREEN 没有置位

则

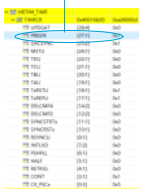
Update 事件不发生，无法触发DAC

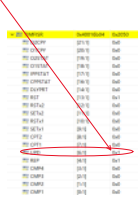


PREEN 置位

则

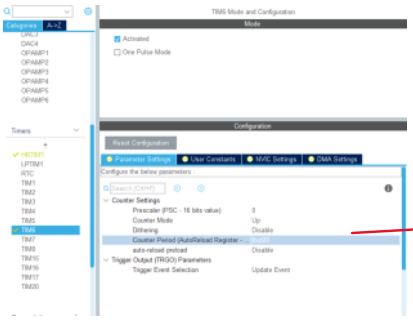
Update 事件发生，触发DAC数据更新

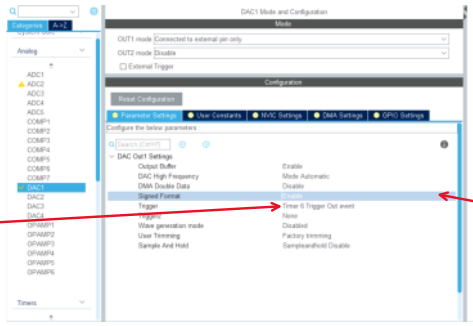




DAC具体配置

2020年4月18日 16:03



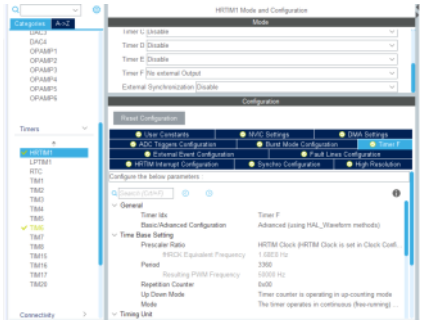


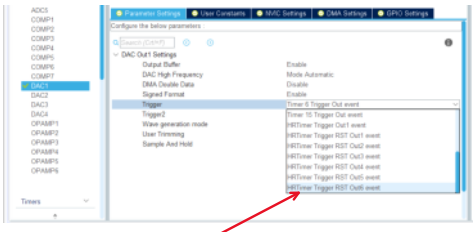
When SINFORMATx bit is set, the MSB bit of the data written to DHRx registers is inverted when it is copied to the DAC_DORx register, and the DAC interface can accept signed data (Q1.15, Q1.11 or Q1.7 format). DAC_DHR12Lx register can be used to store 16-bit signed data in the data holding registers. The 12 MSBs of 16-bit data are used for the DAC output data and the MSB bit is inverted. The four LSBs are simply ignored.

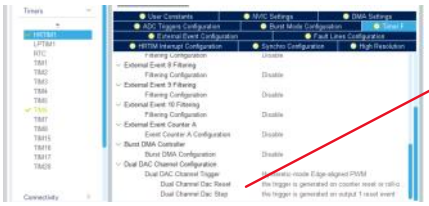
选定有符号位的数据格式

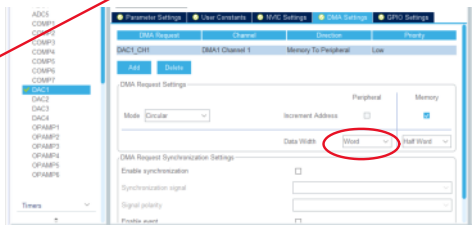
方便使用： arm_float_to_q15 &fsin, &mysin[i], 1);

转换浮点数到整型







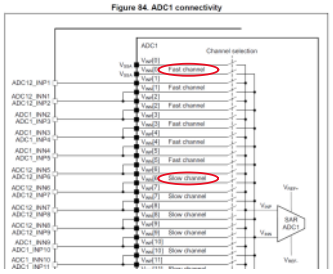


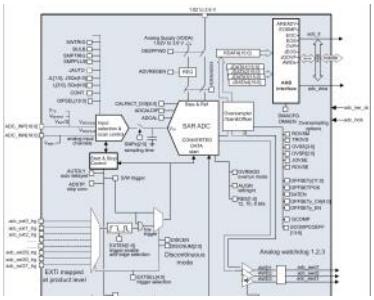
pll

2020年4月19日 7:17

ADC

2020年4月10日 7:17





Regular conversion can then start either by setting ADSTART=1 (refer to Section 21.4.18: Conversion on external trigger and trigger polarity (EXTSEL, EXTEN,JEXTSEL, JEXTEN)) or when an external trigger event occurs, if triggers are enabled.

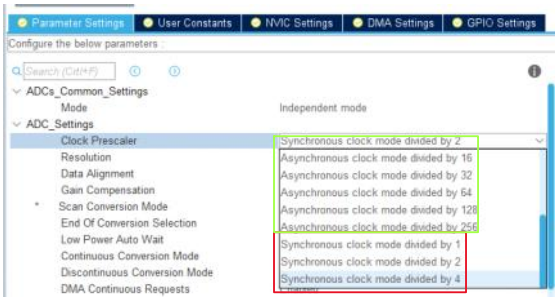
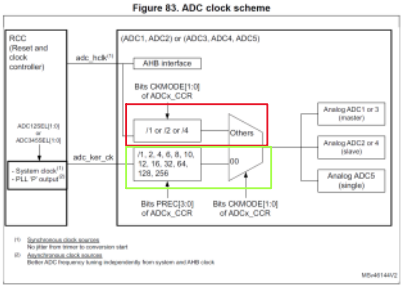
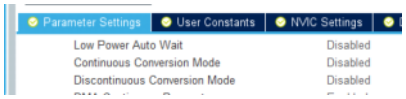


Figure 83. ADC clock scheme





Single conversion mode (CONT=0)

Continuous conversion mode (CONT=1)

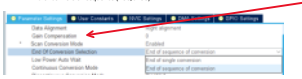
the ADC performs once all the regular conversions of the channels and then automatically restarts and continuously converts each conversions of the sequence.

It is not possible to have both discontinuous mode and continuous mode enabled: it is forbidden to set both DISCEN=1 and CONT=1.

21.4.20 Discontinuous mode

21.4.23 End of conversion, end of sampling phase (EOC, JEOC, EOSMP)

21.4.24 End of conversion sequence (EOS, JEOS)



Gain compensation

When GCOMP bit is set in ADC_CFGR2 register, the gain compensation is activated on all the converted data. After each conversion, data is calculated with the following formula.

DATA = DATA(adc result) × (GCOMPCOEFF) / 4096

As GCOMPCOEFF can be programmed from 0 to 16383, the actual gain compensation factor can range from 0 to 3.999756.

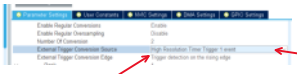
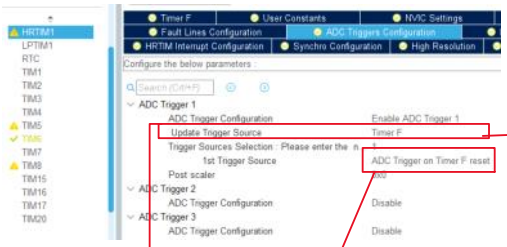


Table 161. Configuring the trigger polarity for regular external triggers

EXTEN[1:0]	Source
00	Hardware Trigger detection disabled, software trigger detection enabled
01	Hardware Trigger with detection on the rising edge
10	Hardware Trigger with detection on the falling edge
11	Hardware Trigger with detection on both the rising and falling edges

Table 163. ADC1/2 - External triggers for regular channels

Name	Source	Type	EXTSEL[4:0]
adc_ext_trg21	hrtim_adc_trg1	Internal signal from on-chip timers	10101



HRTIM_ADC1R to HRTIM_ADC4R and HRTIM_ADCER registers are preloaded and can be updated synchronously with the timer they are related to.
When the preload is disabled (PREEN bit reset) in the source timer, the HRTIM_ADCxR registers are not preloaded either: a write access results in an immediate update of the trigger source.

不同

Figure 251. ADC trigger selection overview

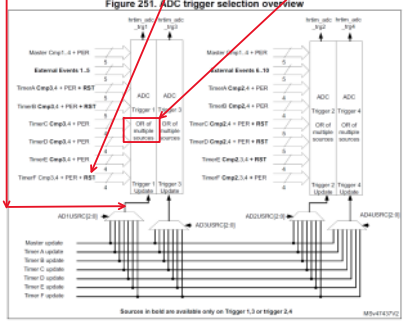
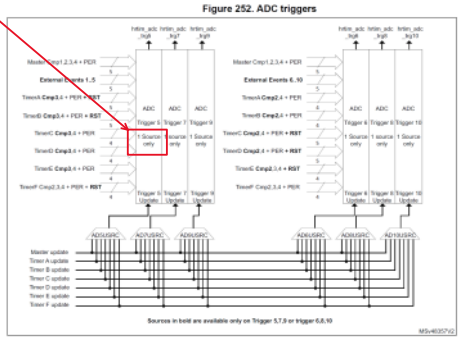
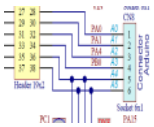
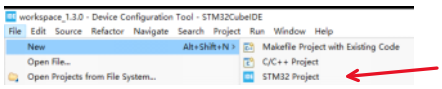


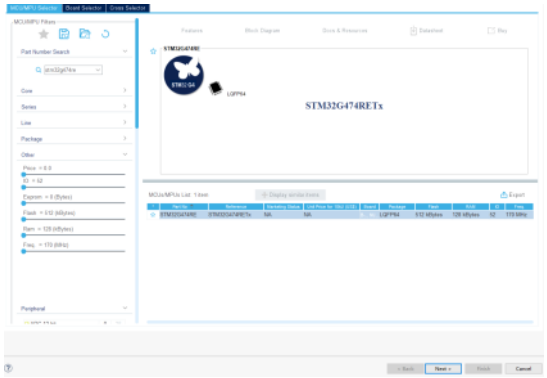
Figure 252. ADC triggers

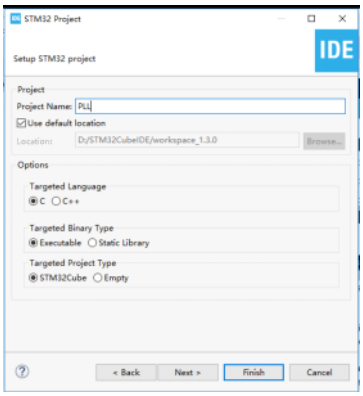


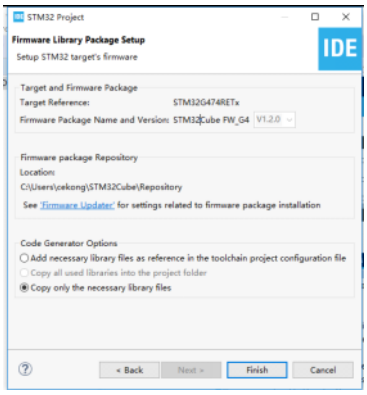

CN8
PA0
PA1
PA4
PB0
PA2
PA5
Header 19x2
Socket 8x1
PA15

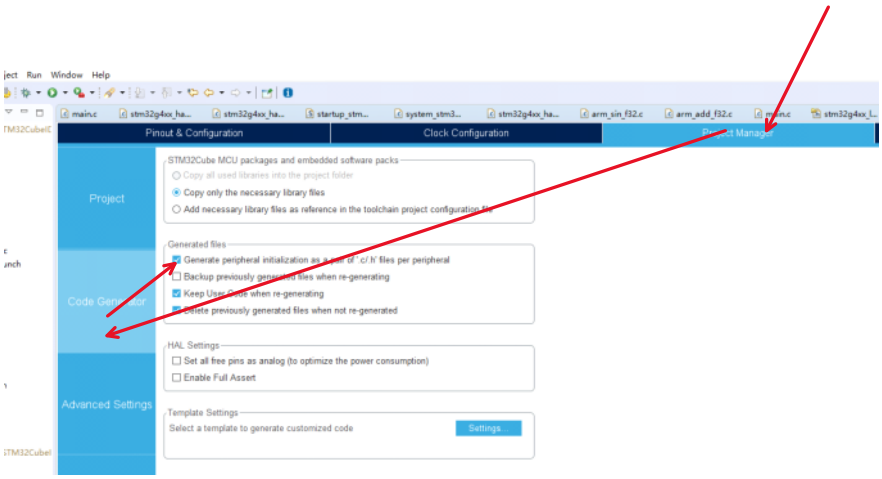

PA3
PA4
ADC1_IN4
DAC1_OUT1

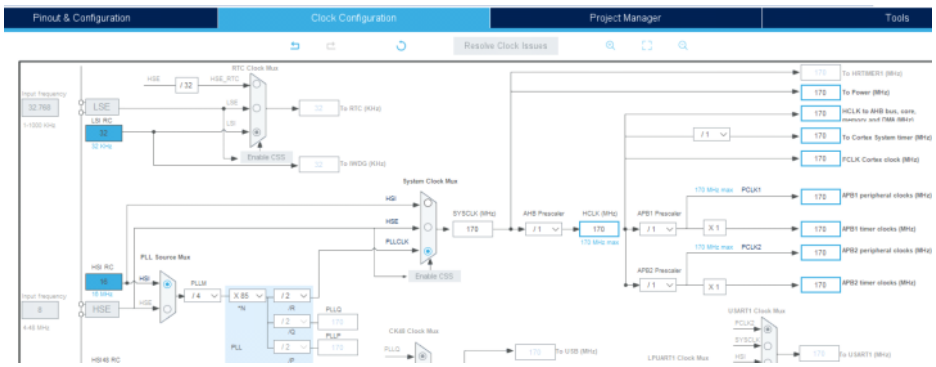

workspace_1.3.0 - Device Configuration Tool - STM32CubeIDE
File Edit Source Refactor Navigate Search Project Run Window Help
New Alt+Shift+N
Open File...
Open Projects from File System...
Makefile Project with Existing Code
C/C++ Project
STM32 Project

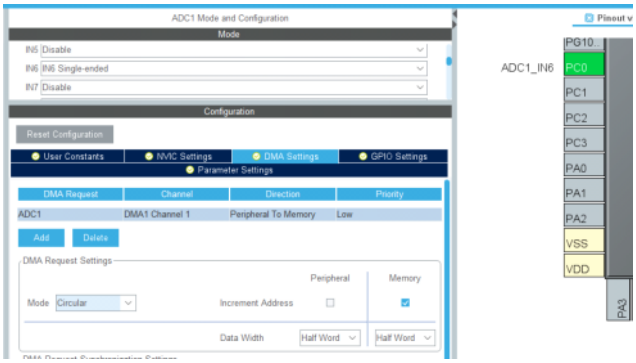

STM32G474RETx

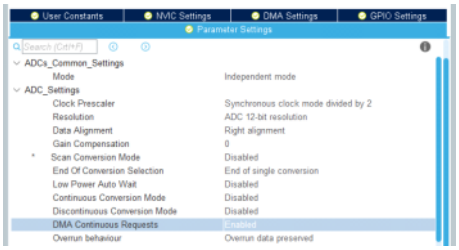

STM32 Project
Setup STM32 project
IDE
Project
Project Name: PLL
Use default location
Location: D:/STM32CubeIDE/workspace_1.3.0
Browse...
Options
Targeted Language
C C++
Targeted Binary Type
Executable Static Library
Targeted Project Type
STM32Cube Empty
< Back
Next >
Finish
Cancel

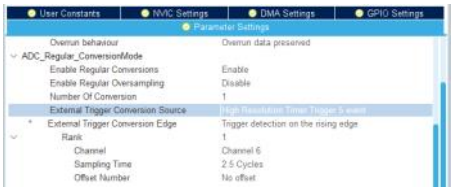

STM32 Project
Firmware Library Package Setup
Setup STM32 target's firmware
IDE
Target and Firmware Package
Target Reference: STM32G474RETx
Firmware Package Name and Version: STM32Cube FW_G4 V1.2.0
Firmware package Repository
Location:
C:\Users\cekong\STM32Cube\Repository
See 'Firmware Updater' for settings related to firmware package installation
Code Generator Options
Add necessary library files as reference in the toolchain project configuration file
Copy all used libraries into the project folder
Copy only the necessary library files
< Back
Next >
Finish
Cancel


Pinout & Configuration
Clock Configuration
Project Manager
Project
Code Generator
Advanced Settings
STM32Cube MCU packages and embedded software packs
Copy only the necessary library files
Add necessary library files as reference in the toolchain project configuration file
Generated files
Backup previously generated files when re-generating
Keep User Code when re-generating
Delete previously generated files when not re-generated
HAL Settings
Set all free pins as analog (to optimize the power consumption)
Enable Full Assert
Template Settings
Select a template to generate customized code
Settings

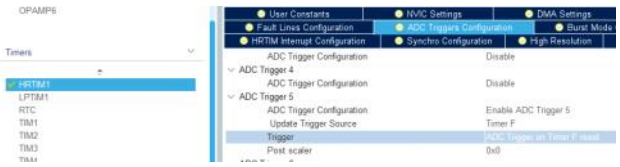

Pinout & Configuration
Clock Configuration
Project Manager
Tools
Resolve Clock Issues


ADC1 Mode and Configuration
Mode
IN5 Disable
IN6 IN6 Single-ended
IN7 Disable
Configuration
Reset Configuration
User Constants
NVIC Settings
DMA Settings
GPIO Settings
Parameter Settings
DMA Request
Channel
Direction
Priority
ADC1
DMA1 Channel 1
Peripheral To Memory
Low
Add
Delete
DMA Request Settings
Peripheral
Memory
Mode Circular
Increment Address
Data Width
Half Word
Pinout v
ADC1_IN6
PG10
PC0
PC1
PC2
PC3
PA0
PA1
PA2
VSS
VDD
PA3

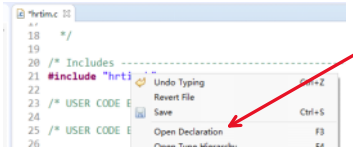

User Constants
NVIC Settings
DMA Settings
GPIO Settings
Parameter Settings
ADCs_Common_Settings
Mode
Independent mode
ADC_Settings
Clock Prescaler
Synchronous clock mode divided by 2
Resolution
ADC 12-bit resolution
Data Alignment
Right alignment
Gain Compensation
Scan Conversion Mode
Disabled
End Of Conversion Selection
End of single conversion
Low Power Auto Wait
Disabled
Continuous Conversion Mode
Disabled
Discontinuous Conversion Mode
Disabled
DMA Continuous Requests
Enabled
Overrun behaviour
Overrun data preserved

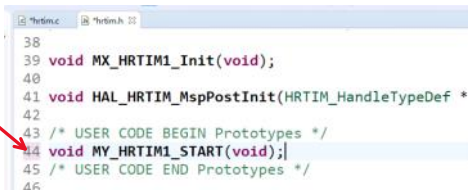

User Constants
NVIC Settings
DMA Settings
GPIO Settings
Parameter Settings
Overrun behaviour
Overrun data preserved
ADC_Regular_ConversionMode
Enable Regular Conversions
Enable
Enable Regular Oversampling
Disable
Number Of Conversion
1
External Trigger Conversion Source
External Trigger Conversion Edge
Trigger detection on the rising edge
Rank
1
Channel
Channel 6
Sampling Time
2.5 Cycles
Offset Number
No offset

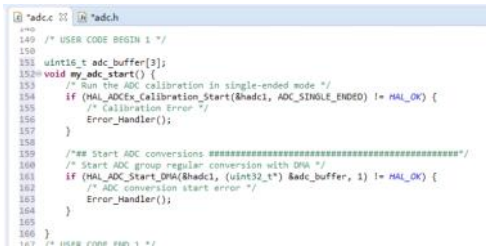

OPAMP5
OPAMP6
Timers
HRTIM1
LPTIM1
RTC
TIM1
TIM2
TIM3
TIM4
TIM5
TIM6
Reset Configuration
User Constants
NVIC Settings
DMA Settings
GPIO Settings
Fault Lines Configuration
ADC Triggers Configuration
Burst Mode Configuration
Timer F
HRTIM Interrupt Configuration
Synchro Configuration
High Resolution
External Event Configuration
Configure the below parameters
General
Timer Idx
Timer F
Basic/Advanced Configuration
Advanced (using HAL_Waveform methods)
Time Base Setting
Prescaler Ratio
HRTIM Clock (HRTIM Clock is set in Clock Configuration Tab wit
HRTIM Equivalent Frequency
1.7E8 Hz
Period
3400
Resulting PWM Frequency
50000 Hz

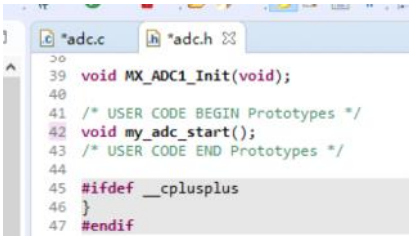

OPAMP6
Timers
HRTIM1
LPTIM1
RTC
TIM1
TIM2
TIM3
User Constants
NVIC Settings
DMA Settings
Fault Lines Configuration
ADC Triggers Configuration
Burst Mode
HRTIM Interrupt Configuration
Synchro Configuration
High Resolution
ADC Trigger Configuration
Disable
ADC Trigger 4
ADC Trigger Configuration
Disable
ADC Trigger 5
ADC Trigger Configuration
Enable ADC Trigger 5
Update Trigger Source
Timer F
Trigger
Post scaler
0x0

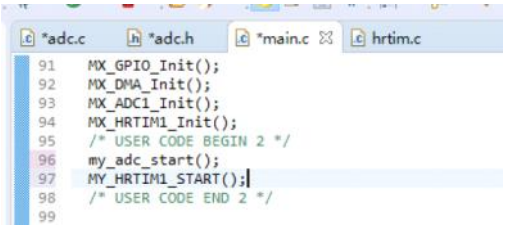

*hrtim.c
180
187 /* USER CODE BEGIN 1 */
188 void MY_HRTIM1_START(void)
189 {
190 HAL_HRTIM_WaveformOutputStart(&hhrtim1, HRTIM_OUTPUT_TF1);
191 HAL_HRTIM_SimpleBaseStart(&hhrtim1,HRTIM_TIMERINDEX_TIMER_F);
192 }
193 /* USER CODE END 1 */
194
195 /********** (C) COPYRIGHT STMicroelectronics *****END OF FILE****/

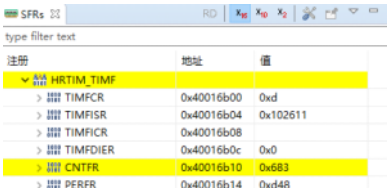

*hrtim.c
18 */
19
20 /* Includes
21 #include "hrti
22
23 /* USER CODE
24
25 /* USER CODE
Undo Typing
Revert File
Save
Ctrl+S
Open Declaration
F3


*hrtim.c
*hrtim.h
38
39 void MX_HRTIM1_Init(void);
40
41 void HAL_HRTIM_MspPostInit(HRTIM_HandleTypeDef *
42
43 /* USER CODE BEGIN Prototypes */
44 void MY_HRTIM1_START(void);
45 /* USER CODE END Prototypes */

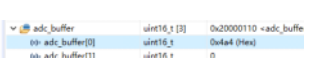

*adc.c
*adc.h
/* USER CODE BEGIN 1 */
uint16_t adc_buffer[3];
void my_adc_start() {
/* Run the ADC calibration in single-ended mode */
if (HAL_ADCEx_Calibration_Start(&hadc1, ADC_SINGLE_ENDED) != HAL_OK) {
/* Calibration Error */
Error_Handler();
}
/*## Start ADC conversions ###*/
/* Start ADC group regular conversion with DMA */
if (HAL_ADC_Start_DMA(&hadc1, (uint32_t*) &adc_buffer, 1) != HAL_OK) {
/* ADC conversion start error */
Error_Handler();
}
}


*adc.c
*adc.h
void MX_ADC1_Init(void);
/* USER CODE BEGIN Prototypes */
void my_adc_start();
/* USER CODE END Prototypes */
#ifdef __cplusplus
}
#endif

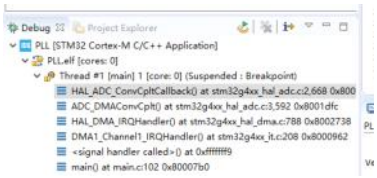

*adc.c
*adc.h
*main.c
hrtim.c
MX_GPIO_Init();
MX_DMA_Init();
MX_ADC1_Init();
MX_HRTIM1_Init();
/* USER CODE BEGIN 2 */
my_adc_start();
MY_HRTIM1_START();
/* USER CODE END 2 */

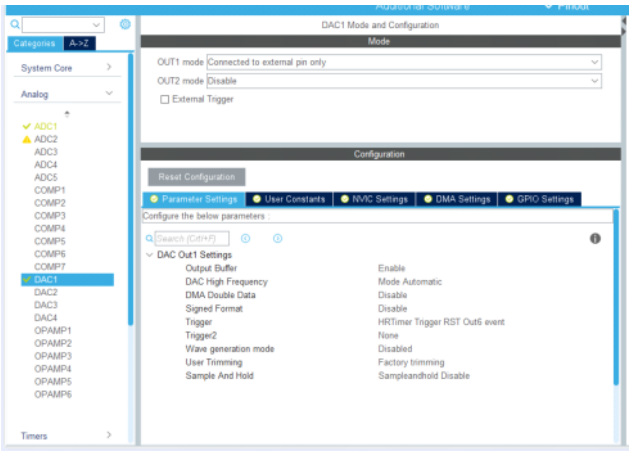

SFRs
type filter text
注册
地址
值
HRTIM_TIMF
TIMFCR 0x40016b00 0xd
TIMFISR 0x40016b04 0x102611
TIMFICR 0x40016b08
TIMFDIER 0x40016b0c 0x0
CNTFR 0x40016b10 0x683

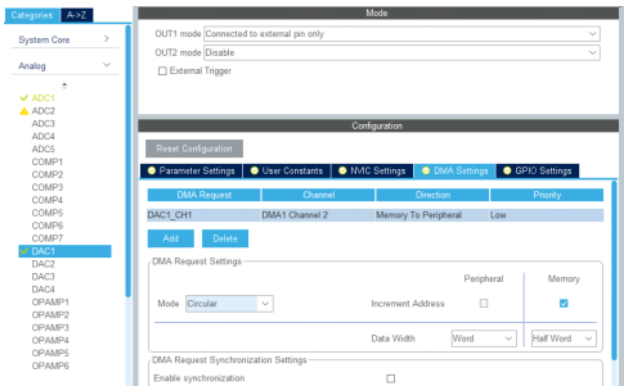

注册
地址
值
ADC1
ISR 0x50000000 0xb
IER 0x50000004 0x10
CR 0x50000008 0x10000005
CFGR 0x5000000c 0x800006e3
CFGR2 0x50000010 0x0
SMPR1 0x50000014 0x0
SMPR2 0x50000018 0x0
TR1 0x50000020 0xff0000
TR2 0x50000024 0xff0000
TR3 0x50000028 0xff0000
SQR1 0x50000030 0x180
SQR2 0x50000034 0x0
SQR3 0x50000038 0x0
SQR4 0x5000003c 0x0
DR 0x50000040 0x4a4

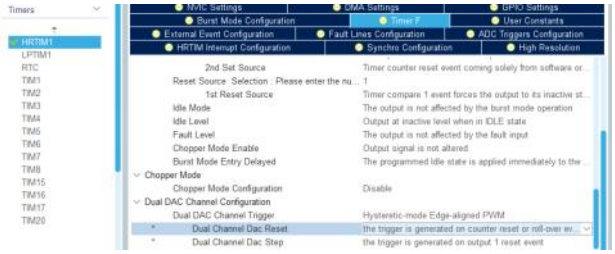

adc_buffer
uint16_t [3]
0x20000110 <adc_buffe
adc_buffer[0]
uint16_t
0x4a4 (Hex)


__weak void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc)
{
/* Prevent unused argument(s) compilation warning */
UNUSED(hadc);
/* NOTE : This function should not be modified. When the callback is needed,
function HAL_ADC_ConvCpltCallback must be implemented in the user file.
*/


Debug
Project Explorer
PLL [STM32 Cortex-M C/C++ Application]
PLL.elf [cores: 0]
Thread #1 [main] 1 [core: 0] (Suspended : Breakpoint)
HAL_ADC_ConvCpltCallback() at stm32g4xx_hal_adc.c:2,668 0x800
ADC_DMAConvCplt() at stm32g4xx_hal_adc.c:3,592 0x8001dfc
HAL_DMA_IRQHandler() at stm32g4xx_hal_dma.c:788 0x8002738
DMA1_Channel1_IRQHandler() at stm32g4xx_it.c:208 0x8000962
<signal handler called>() at 0xffffffff9
main() at main.c:102 0x80007b0


DAC1 Mode and Configuration
Mode
OUT1 mode Connected to external pin only
OUT2 mode Disable
External Trigger
Configuration
Reset Configuration
Parameter Settings
User Constants
NVIC Settings
DMA Settings
GPIO Settings
Configure the below parameters :
DAC Out1 Settings
Output Buffer Enable
DAC High Frequency Mode Automatic
DMA Double Data Disable
Signed Format Disable
Trigger HRTimer Trigger RST Out6 event
Trigger2 None
Wave generation mode Disabled
User Trimming Factory trimming
Sample And Hold Sampleandhold Disable


Mode
OUT1 mode Connected to external pin only
OUT2 mode Disable
External Trigger
Configuration
Reset Configuration
Parameter Settings
User Constants
NVIC Settings
DMA Settings
GPIO Settings
DMA Request
Channel
Direction
Priority
DAC1_CH1
DMA1 Channel 2
Memory To Peripheral
Low
Add
Delete
DMA Request Settings
Mode Circular
Increment Address
Data Width
Word
Half Word
DMA Request Synchronization Settings
Enable synchronization


Timers
HRTIM1
LPTIM1
RTC
TIM1
TIM2
TIM3
TIM4
TIM5
TIM6
TIM7
TIM8
TIM15
TIM16
TIM17
TIM20
NVIC Settings
DMA Settings
GPIO Settings
Burst Mode Configuration
Timer F
User Constants
External Event Configuration
Fault Lines Configuration
ADC Triggers Configuration
HRTIM Interrupt Configuration
Synchro Configuration
High Resolution
2nd Set Source
Timer counter reset event coming solely from software or...
Reset Source Selection : Please enter the nu...
1
1st Reset Source
Timer compare 1 event forces the output to its inactive st...
Idle Mode
The output is not affected by the burst mode operation
Idle Level
Output at inactive level when in IDLE state
Fault Level
The output is not affected by the fault input
Chopper Mode Enable
Output signal is not altered
Burst Mode Entry Delayed
The programmed idle state is applied immediately to the ...
Chopper Mode
Chopper Mode Configuration
Disable
Dual DAC Channel Configuration
Dual DAC Channel Trigger
Hysteretic-mode Edge-aligned PWM
Dual Channel Dac Reset
the trigger is generated on counter reset or roll-over ev...
Dual Channel Dac Step
the trigger is generated on output 1 reset event

输入信号预处理

2020年4月20日　　12:06

低通滤波，获取直流偏置，可以在更新输出后运算

Discrete Transfer Function

Bilinear Transformation with Prewarping
Sample Frequency = 50.00 KHz

$$\frac{6.279e\text{-}04*Z + 6.279e\text{-}04}{Z - .9987}$$

1st Order Low Pass Butterworth
Pass Band Frequency = 10.00 Hz

Mon Apr 20 12:00 2020

$$H(z) = \frac{C(k)}{R(k)} = \frac{6.28e\text{-}4(1+z^{-1})}{1-0.9987z^{-1}}$$

$$C(k) = 6.28e^{-4}(1+z^{-1})R(k) - \frac{(-0.9987)C(k-1)}{4}$$

误差累积，不会收敛 C(k)发散吗

6

运算速度提升

2020年4月21日 9:38

三相SPLL C代码 硬件（FMAC, CORDIC) 加速

2020年4月22日 7:12

运算定点化

2020年4月21日 9:38

程序在CCM内存运行

2020年4月21日 9:39

https://community.st.com/s/question/0D50X00009XkYIO/how-to-use-ccm-ram-within-stm32-microcontroller-for-functions-and-variables

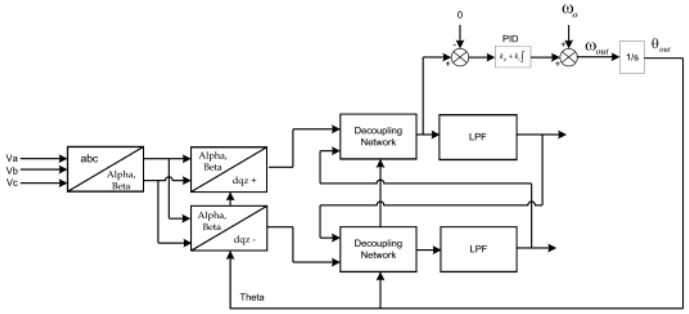
www.st.com

http://www.st.com/resource/en/application_note/dm00083249.pdf

三相SPLL C代码分析

2020年4月21日 10:09

```
//全局变量声明                                                    支持文件
ABC_DQ0_POS     TINV_abc_dq0_pos_vG;                            abc_dq0_pos.h
ABC_DQ0_NEG     TINV_abc_dq0_neg_vG;                            abc_dq0_neg.h

                                                                spll_3ph_ddsrf.h

static inline void TINV_runISR1(void)
{

  ABC_DQ0_POS_run(&TINV_abc_dq0_pos_vG, TINV_vGA_sensed, TINV_vGB_sensed, TINV_vGC_sensed,   TINV_acSine,INV_acCosine);
  ABC_DQ0_NEG_run(&TINV_abc_dq0_neg_vG, TINV_vGA_sensed, TINV_vGB_sensed, TINV_vGC_sensed,  TINV_acSine, TINV_acCosine);


  SPLL_3ph_DDSRF_run(&TINV_spll_3ph_1,     TINV_abc_dq0_pos_vG.d, TINV_abc_dq0_neg_vG.d,   TINV_abc_dq0_pos_vG.q, TINV_abc_dq0_neg_vG.q);
  TINV_acSine = sinf(TINV_spll_3ph_1.theta[1]);
  TINV_acCosine = cosf(TINV_spll_3ph_1.theta[1]);
```



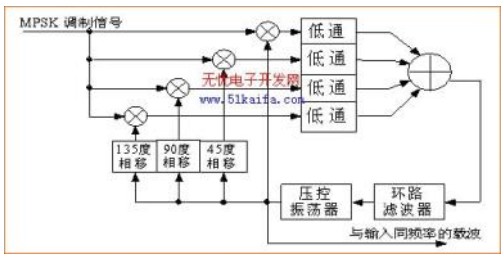

```
  ABC_DQ0_POS_reset(&TINV_abc_dq0_pos_vG);
  ABC_DQ0_NEG_reset(&TINV_abc_dq0_neg_vG);


  SPLL_3ph_DDSRF_init(TINV_AC_FREQ,
                      (float32_t)(1.0 / TINV_ISR_CONTROL_FREQUENCY),
                      (float32_t)(0.00188141f),
                      (float32_t)(-0.99623717f),
                      &TINV_spll_3ph_1);

  TINV_spll_3ph_1.lpf_coeff.b0 = 333.807f;
  TINV_spll_3ph_1.lpf_coeff.b1 = -333.674f;
```

```
//! \brief          Initialize SPLL_3ph_DDSRF module
//! \param grid_freq    The grid frequency
//! \param delta_t     1/Frequency of calling the PLL routine
//! \param k1          parameter
//! \param k2          parameter
//! \param spll_obj    The SPLL_3ph_DDSRF structure
//!
static inline void SPLL_3ph_DDSRF_init(float grid_freq, float delta_t,
                                       float k1, float k2,
                                       SPLL_3ph_DDSRF *spll_obj)
{
    spll_obj->d_p_decoupl=(float)(0.0);
    spll_obj->d_n_decoupl=(float)(0.0);

    spll_obj->q_p_decoupl=(float)(0.0);
    spll_obj->q_n_decoupl=(float)(0.0);

    spll_obj->d_p_decoupl_lpf=(float)(0.0);
    spll_obj->d_n_decoupl_lpf=(float)(0.0);

    spll_obj->q_p_decoupl_lpf=(float)(0.0);
    spll_obj->q_n_decoupl_lpf=(float)(0.0);

    spll_obj->y[0]=(float)(0.0);
    spll_obj->y[1]=(float)(0.0);

    spll_obj->x[0]=(float)(0.0);
    spll_obj->x[1]=(float)(0.0);

    spll_obj->w[0]=(float)(0.0);
    spll_obj->w[1]=(float)(0.0);

    spll_obj->z[0]=(float)(0.0);
    spll_obj->z[1]=(float)(0.0);

    spll_obj->k1=k1;
    spll_obj->k2=k2;

    spll_obj->v_q[0]=(float)(0.0);
    spll_obj->v_q[1]=(float)(0.0);

    spll_obj->ylf[0]=(float)(0.0);
    spll_obj->ylf[1]=(float)(0.0);

    spll_obj->fo=(float)(0.0);
    spll_obj->fn=(float)(grid_freq);

    spll_obj->theta[0]=(float)(0.0);
    spll_obj->theta[1]=(float)(0.0);

    spll_obj->delta_t= delta_t;
}
```

```
//! \brief          Run spll_3ph_srf module
//! \param spll_obj    The spll_3ph_ddsrf structure
//! \param d_p                  D Positive seq component of the grid voltage
//! \param d_n                  D Negative seq component of the grid voltage
//! \param q_p                  Q Positive seq component of the grid voltage
//! \param q_n                  Q Negative seq component of the grid voltage
//!
static inline void SPLL_3ph_DDSRF_run(SPLL_3ph_DDSRF *spll_obj,
                                      float d_p, float d_n,
                                      float q_p, float q_n)
{
    // before calling this routine run the ABC_DQ0_Pos & Neg run routines
  // pass updated values for d_p,d_n,q_p,q_n
    // and update the cos_2theta and sin_2theta values with the prev angle

    //------------------------//
    // Decoupling Network     //
    //------------------------//
    spll_obj->d_p_decoupl= d_p  - (spll_obj->d_n_decoupl_lpf*spll_obj->cos_2theta)  - (spll_obj->q_n_decoupl   *spll_obj->sin_2theta);
    spll_obj->q_p_decoupl= q_p  + (spll_obj->d_n_decoupl_lpf*spll_obj->sin_2theta)  - (spll_obj->q_n_decoupl   *spll_obj->cos_2theta);

    spll_obj->d_n_decoupl= d_n  - (spll_obj->d_p_decoupl_lpf*spll_obj->cos_2theta)  + (spll_obj->q_p_decoupl   *spll_obj->sin_2theta);
    spll_obj->q_n_decoupl=q_n   - (spll_obj->d_p_decoupl_lpf*spll_obj->sin_2theta)  - (spll_obj->q_p_decoupl   *spll_obj->cos_2theta);

    //------------------------//
    // Low pass filter        //
    //------------------------//

    spll_obj->y[1]=(spll_obj->d_p_decoupl*spll_obj->k1) - (spll_obj->y[0]*spll_obj->k2);
    spll_obj->d_p_decoupl_lpf=spll_obj->y[1] + spll_obj->y[0];
    spll_obj->y[0]=spll_obj->y[1];

    spll_obj->x[1]=(spll_obj->q_p_decoupl*spll_obj->k1) - (spll_obj->x[0]*spll_obj->k2);
    spll_obj->q_p_decoupl_lpf=spll_obj->x[1] + spll_obj->x[0];
    spll_obj->x[0]=spll_obj->x[1];

    spll_obj->w[1]=(spll_obj->d_n_decoupl*spll_obj->k1) - (spll_obj->w[0]*spll_obj->k2);
    spll_obj->d_n_decoupl_lpf=spll_obj->w[1] + spll_obj->w[0];
    spll_obj->w[0]=spll_obj->w[1];

    spll_obj->z[1]=(spll_obj->q_n_decoupl*spll_obj->k1) - (spll_obj->z[0]*spll_obj->k2);
    spll_obj->q_n_decoupl_lpf=spll_obj->z[1] + spll_obj->z[0];
    spll_obj->z[0]=spll_obj->z[1];

    spll_obj->v_q[0]=spll_obj->q_p_decoupl;

    //------------------------------//
    // Loop Filter                  //
    //------------------------------//
    spll_obj->ylf[0]=spll_obj->ylf[1]  + (spll_obj->lpf_coeff.b0*spll_obj->v_q[0])  + (spll_obj->lpf_coeff.b1*spll_obj->v_q[1]);
    spll_obj->ylf[1]=spll_obj->ylf[0];
    spll_obj->v_q[1]=spll_obj->v_q[0];

    //------------------------------//
    // VCO                          //
    //------------------------------//
    spll_obj->fo=spll_obj->fn+spll_obj->ylf[0];

    spll_obj->theta[0]=spll_obj->theta[1] +  ((spll_obj->fo*spll_obj->delta_t)*(float)(2.0*3.1415926));
    if(spll_obj->theta[0]>(float)(2.0*3.1415926))
        spll_obj->theta[0]=spll_obj->theta[0] - (float)(2.0*3.1415926);

    spll_obj->theta[1]=spll_obj->theta[0];

    spll_obj->cos_2theta=cosf(spll_obj->theta[1]*2.0);
    spll_obj->sin_2theta=sinf(spll_obj->theta[1]*2.0);
}
```

MPSK 载波恢复

2020年8月6日　　21:09



https://www.eefocus.com/component/470460

AC7010与AC7020区别		
	AC7010	AC7020
FPGA 型号	XC7Z010-1CLG400C	XC7Z020-2CLG400I
速度等级	-1	-2
芯片级别	商业级	工业级
内存大小	4Gbit	8Gbit
逻辑单元数量	28K	85K
ARM 核主频	666MHz	767MHz

锁相环有关文献线索

2022年12月28日 18:02

10.1109/JESTPE.2014.2345741

多篇讨论PLL的文献引用此文